2026年QG前端招新最终考核题目

💡 考核须知

1. 请阅读所有题目后仔细斟酌选题。**并非完成难度高的题目即得分高**，最终得分视功能完成情况、拓展功能、代码风格、**报告文档和PPT演示（占 30%）**而定。
2. 交付内容：**演示视频**（有时间可以加上字幕等，尽可能清晰展示）+ **报告文档**（报告文档**包括但不限于**：项目简介，设计思路，详细功能设计，项目亮点，心得体会）+ **PPT演示**
3. 可以自行扩充或者新增功能，视情况而定加分。
4. 尽量避免出现Bug，不要为了多做功能而增加bug，会重点关注项目的**健壮性**
5. 技术栈的选取需**严格按照每个题目的要求**
6. !!结合自己的技术能力正确地使用AI工具 !!

> 🤩 **请尽情展示你的技术宽度和技术深度吧！！！一定要坚持到最后呀！！**

题目一

一、考题背景

一套适配多角色使用的教学管理系统，满足成绩录入、查询、统计、班级通知，操作日志等核心需求，要求基于原生技术栈完成该系统的全流程开发，重点考察前后端协同能力，架构设计能力，复杂业务理解和处理能力及问题兜底处理能力。

二、考题总体要求

（一）技术栈要求

- 要求使用 HTML/CSS/JS + Node.js + MySQL **原生技术栈**，不得使用第三方库等

（二）总体要求

1. 开发规范：代码书写规范、注释清晰易懂，无全局变量污染问题，核心业务逻辑具备可复用性；
2. 数据安全：实现接口简单鉴权、SQL 防注入机制，确保系统数据存储和交互安全；
3. 跨端适配：支持 PC、手机全终端适配，交互流程流畅，性能优化措施到位；

4. 系统稳定：实现操作行为追溯、异常场景兜底处理，确保系统无阻断性运行问题；
5. 消息管理：实现班级定向通知、已读未读状态区分、已读人数统计，消息数据与班级、用户强关联。

三、考题要求实现的基本功能

1. 功能层面

（1）登录与权限管控

- 实现简易登录功能，支持 JWT 或 Cookie 鉴权方式，登录态持久化存储；
- 未登录状态下拦截所有功能操作，自动跳转至登录页面；
- 实现「教务主任」「班主任」「普通学生」三类角色权限管控：
  - 教务主任：拥有全功能权限，不受跨班级访问限制，包含班级管理、成绩增删改查、日志查看、数据导出、**查看全量班级通知，完成教师与班级的绑定操作**；
  - 班主任：仅可操作所属班级成绩（查询、修改、统计），不可查看全量操作日志，只可以查看班级操作日志；**可发布所属班级通知、编辑/删除本班通知、查看通知已读人数统计**；
  - 普通学生：仅可查询本人成绩及班级统计数据，无任何修改、删除权限；**仅可查看所属班级通知、标记消息已读**。

（2）班级管理

- 支持教导主任对班级和教师的绑定操作，设置合理的绑定/解绑规则防止信息混乱；
- 支持新增班级，补录班级成员，删除班级成员；

（3）成绩操作

- 支持单条成绩添加、批量成绩添加功能；
- 可按班级、科目、学号等条件查询、修改、删除成绩；
- 成绩操作后实时刷新成绩列表及统计数据，支持成绩列表批量导出。

（4）多维度统计

- 可按班级、科目维度筛选统计成绩数据；
- 展示维度包含平均分、最高分、最低分、及格人数、及格率，数据实时更新。

（5）⭐⭐日志查看

- 支持分页查看操作日志，清晰展示操作人、操作类型、操作内容、操作时间；
- 日志内容展示范围与当前登录角色权限匹配，仅展示对应权限可见内容；
- **通知操作日志：记录通知发布、编辑、删除行为。**

（6）⭐⭐班级通知平台

- **通知发布（班主任专属）**：班主任仅可向**自己所属班级**发布通知，支持填写标题、内容、发布时间，系统自动关联发布人与班级；
- **通知编辑与删除（班主任专属）**：支持修改/删除本班未过期通知，操作后同步更新学生端展示；
- **已读统计（班主任专属）**：实时展示单条通知的**已读人数、未读人数、总人数**，可查看已读/未读学生名单；
- **学生端通知展示**：学生仅可查看**自己所属班级**的通知列表，通知自动区分「已读/未读」状态，未读消息高亮标记；
- **已读标记**：学生点击/查看通知后，自动标记为已读，同步更新后端已读状态；
- **通知列表**：支持按时间倒序排列，支持分页加载，未读消息优先展示；
- **权限拦截**：禁止跨班级查看/操作通知，学生无法发布/编辑通知，教务主任可查看所有班级通知。

2. 布局外观层面

（1）页面区域划分

- 要求页面布局有良好的划分，页面简洁美观。

（2）响应式适配

- 大屏（≥1200px）：采用多区域网格布局，班级 / 筛选条件横向排列，列表区固定表头，**通知区与成绩区左右分栏展示**；
- 中屏（768px - 1199px）：筛选条件折叠为下拉框，列表区适配页面宽度，**通知区独立卡片展示**；
- 小屏（<768px）：所有区域垂直排列，表单元素 100% 宽度展示，表格支持横向滚动，筛选区折叠为弹窗，按钮组适配移动端点击体验，**通知列表全屏展示，未读角标醒目**。

（3）样式优化

- 使用 CSS 变量统一页面主题色；
- 表格设置奇偶行区分样式；
- 加载状态展示骨架屏，替代纯文字提示；
- **通知样式优化**：未读消息红点/角标提醒、已读消息置灰区分，通知卡片支持展开/收起详情。

3. 安全层面

- 实现接口简单鉴权机制，未授权请求无法访问核心接口；
- 后端数据库操作采用防 SQL 注入措施，避免恶意注入攻击；
- 操作日志完整记录系统行为，支持行为追溯；
- 数据校验覆盖前端初步校验、后端严格校验，确保数据合法性；

- 通知安全：通知数据与班级ID、用户ID强绑定，防止越权查看/篡改他人班级通知。

四、⭐⭐⭐加分点

1. **前端组件封装**：封装弹窗、表单、表格、**通知卡片**等通用组件，基于原型 / 类实现，支持组件复用和扩展；
2. **本地缓存优化**：缓存班级列表、当前用户权限、最近筛选条件、**未读通知列表**等常用数据，离线时展示缓存数据，联网后自动同步更新；
3. **通知体验优化**：实现通知未读红点实时刷新、新通知桌面提醒、班主任端已读人数实时刷新。
4. **实现 SPA 单页应用**：基于原生 JavaScript 实现前端路由切换，无刷新页面跳转，提升系统操作流畅度与用户体验；
5. **列表性能优化**：针对大数据量列表采用分页查询/虚拟滚动等方式，提升大数据量下的列表加载、滚动、筛选操作性能，降低页面卡顿与内存占用。
6. **请求与操作优化**：前端拦截同一时间多次提交的重复请求，避免无效接口调用与数据冗余；对成绩筛选、分页切换、通知搜索等高频操作添加防抖处理，有效防止用户多次误触。

五、验收标准

1. 系统运行：系统可正常启动，无语法错误、运行崩溃问题，核心业务流程无阻断；
2. 功能实现：所有基本功能可正常运行，流程连贯，数据交互正常，角色权限管控生效；
3. 跨端适配：页面适配 PC、手机全终端，样式无错乱，交互体验良好，加载状态反馈清晰；
4. 代码规范：代码书写规范，模块化拆分清晰，具备良好的项目目录结构，可维护性强；
5. 安全与兜底：数据安全、一致性有保障，网络中断、参数错误、权限越界等异常情况有兜底处理，提示信息友好；通知越权访问、数据异常有完善兜底。

题目二

QG团队协作与版本管理平台

一、核心技术规范

1. 前端页面所有功能需使用原生HTML、CSS、Javascript实现，除非特别声明，不可使用第三方插件、组件库（如 Vue/React）或框架。数据若无特别声明则统一自行拟造。
2. 后台功能采用 Node.js 开发，最好使用纯原生模块开发（除 mysql、mongodb 等数据库模块），也可以使用 express 等框架。使用原生加分。
3. 采用 MVC 模式（前后端分离），数据存储使用 MySQL 或 MongoDB。

二、项目背景

在团队项目开发中，协作与版本控制是确保项目高效推进和避免冲突的核心要素。为了更好地管理任务进度、分配任务、跟踪历史版本、保证团队间的无缝协作，我们计划开发一个类似于GitHub的团队协作平台。这个平台不仅仅是任务管理工具，还可以进行分支管理和历史记录追溯，确保每个小组成员在最合适的时间点参与任务并及时获取更新。

三、核心考察点

1. 任务管理 / 分支管理 / 历史追溯
2. 团队小组管理
3. 前端交互设计
4. 纯前端分支模拟 + 冲突解决
5. 事件委托、状态管理
6. 原生组件化思维

四、主体必做功能（10项）

4. 用户注册与登录

基础功能

- 注册/登录：支持邮箱/手机号注册，使用**Token/Cookie**身份验证
- 角色体系：
  - 管理员：创建小组、分配任务、管理分支、管理历史版本
  - 普通用户：查看/更新个人任务、参与分支合并

现代原生强制要求

- 前端实现**密码强度即时反馈动画**
- 依据后端角色**动态渲染/销毁受保护DOM**（如“解决冲突”按钮）
- 实现**hashchange前端路由守卫**，防止越权访问

5. 团队与小组管理

基础功能

- 小组管理：管理员创建小组，支持自定义名称、描述、头像
- 任务分配：管理员分配任务，指定负责人、优先级、状态标签
- 成员协作：成员可查看全组任务，仅可操作个人负责任务，支持任务评论

现代原生强制要求

- 任务列表必须使用**事件委托（Event Delegation）**
- 禁止在循环中为每个任务卡片单独绑定事件
- 防止DOM内存泄漏

6. 任务管理与分支管理

任务必填字段

1. 标题
2. 描述
3. 状态（待办/进行中/已完成）
4. 优先级（低/中/高）
5. 标签（bug修复/功能开发/优化等）
6. 负责人
7. 分支列表
8. 进度条（完成百分比）
9. 截止日期
10. 任务评论

现代原生强制要求

- 遵循**数据驱动视图**
- 状态/优先级变更：通过重新渲染过滤后的数据数组更新视图，禁止单独修改DOM节点

7. 历史记录与版本追溯

基础功能

- 任务/分支历史记录：记录操作类型、操作人、描述、时间
- 版本回退：管理员可回退任务至历史版本
- 分支历史：记录创建/修改/合并操作

现代原生强制要求

- 切换分支/版本回退时，实现**前端脏状态拦截**
- 检测到未保存修改：弹窗确认
- 确认后**JS动态替换DOM**，严禁使用 `location.reload()` 刷新页面

8. 任务合并与冲突管理（核心难点）

基础功能

- 分支合并：成员提交合并请求 → 管理员审核合并
- 冲突管理：系统自动检测冲突，展示冲突内容，管理员处理

现代原生强制要求

- 前端先调用**合并检测接口**
- 检测到文本冲突：**动态生成冲突解决编辑器**（高亮 `<<<<<<<` 等标记）
- 强制管理员**前端手动修改清洗文本**后，再提交后端完成合并

9. 通知与实时更新

基础功能

- 实时通知：任务更新、合并请求、评论自动推送
- 实时更新：无需手动刷新，自动同步任务进展

现代原生强制要求

- 推荐方案：AJAX短轮询 / SSE
- 使用**Set数据结构**实现消息/任务ID去重
- 避免重复渲染、DOM闪烁

10. 任务依赖管理

基础功能

- 支持创建任务依赖关系
- 依赖任务完成后，自动通知下一任务负责人

现代原生强制要求

- 提交依赖前，前端实现**DFS图算法**
- 拦截**循环依赖**（如A依赖B、B依赖A），避免死循环

五、加分项（原生JS深度展示）

11. 原生富文本编辑器

- 禁止第三方插件
- 基于 `contenteditable="true"` 实现

- 支持：加粗、斜体、插入代码块

12. 纯前端数据导出

- 支持导出：Excel/CSV/PDF进度报告
- 原生实现：`Blob` + `URL.createObjectURL`
- 无后端I/O负载

13. 多设备适配 + 骨架屏

- 响应式布局，适配PC/移动端
- 数据加载前：**CSS原生骨架屏动画**
- 替代“加载中”文字

14. 多维标签筛选

- 支持多标签交叉筛选
- 使用数组高阶函数：`filter` / `reduce`
- 渲染使用 `DocumentFragment` 优化DOM性能

15. 实时语音/文字聊天

- 文字聊天：封装原生WebSocket
- 语音聊天（超高加分）：`MediaRecorder` 获取麦克风流，传输/播放Blob

16. 时间追踪 + 自定义工作流

- 自动记录任务工作时长，生成统计报告
- 支持自定义任务流程（待办→进行中→测试→完成）

17. 桌面级系统通知

- 使用浏览器原生**Notification API**
- 跨标签页桌面弹窗提醒
- 任务截止、进度更新自动推送

18. 动态优先级调整

- 依据截止时间自动计算优先级
- 使用 `requestAnimationFrame` /定时器
- 动态变更任务卡片警戒色

六、大加分项（硬核前端挑战）

19. 沉浸式看板（Kanban）+ 乐观更新

看板交互

- 手写**HTML5 Drag and Drop API**
- 支持拖拽卡片更新任务状态
- 拖拽目标区域虚线高亮反馈

乐观更新

- 拖拽后**前端立即更新UI**
- 异步发送后端请求
- 请求失败（断网等）**平滑回滚UI状态**

20. 原生历史图表 + 动态进度条

- 进度条：CSS3平滑过渡动画，自动更新比例
- 历史图表：**原生Canvas/SVG**手绘
- 禁止ECharts等第三方库
- 绘制状态时间线/折线图，手动计算坐标轴

题目三

课题目标

本次课题旨在通过从零到一构建一个 MVP（Minimum Viable Product，最小可行产品）级别的画布应用，让你综合运用前后端知识，深入理解图形编辑器的核心原理与工程挑战。你将直面图形渲染、交互设计、性能优化、数据持久化等真实场景，并有机会探索协同编辑等前沿领域。

我们期望你交付一个**稳定、流畅、可解释**的作品，并在过程中锻炼架构设计、技术取舍与复杂问题拆解的能力。

技术选型与范围

框架与渲染层

- **前端框架**：你可以在 `React` 或 `Vue` 中自由选择，也可以选择前端三剑客，建议依据个人的熟悉度来定。
- **渲染层**：我们鼓励你探索 `Canvas` 与 `DOM` 结合的渲染方案。

- **取舍建议**：你可以思考并阐述你的选择。例如，使用 `Canvas` 承载核心画布区域以追求极致性能和绘制自由度，同时利用 `DOM` 构建上层 UI（如工具栏、属性面板），实现开发效率与性能的平衡。在说明文档中，请简要解释你选择的渲染方案及其背后的考量。

核心库使用规范

为保证课题的核心挑战性，我们对第三方库的使用做出如下约定：

- **禁止使用**：严禁使用任何高度封装的图形库或框架，如 `react-flow`、`tldraw`、`konva`、`fabric.js` 等。直接使用这类库将导致作业无效。
- **允许使用**：
  - **基础渲染库**：允许使用 `Pixi.js` 等底层图形渲染库，辅助你更高效地与 `WebGL/Canvas` API 交互。
  - **局部能力库**：在非核心业务逻辑上，可使用成熟的三方库解决特定问题（例如，使用 `prosemirror` 或 `slate.js` 实现富文本编辑）。
  - **AI 辅助工具**：允许使用 `Cursor`、`Claude code` 等 AI 编程助手。善用 AI 是重要的工程能力，我们鼓励你借此提升任务拆分和架构设计水平，但请务必保证对生成代码的理解与掌控。

核心原则：我们希望你自己实现画布的核心业务逻辑，包括但不限于：元素管理、视口变换、选区计算、事件处理、组合/解组等。在说明文档中，请明确你使用了哪些第三方库，以及它们承担的职责边界。

功能清单：从 MVP 到完整实现

我们将功能要求划分为 **P0（MVP 必做）** 和 **P1（进阶挑战）**。评审将更看重 P0 部分的完成度和质量。挑战项（⭐）代表了从基础到高阶的探索方向，完成度越高，加分越多。

P0 级：核心功能 (MVP)

前端部分

分类	功能点	说明与要求
基础渲染	图形渲染	• 至少支持3种基础图形（如矩形、圆形、三角形） • 支持设置背景色、边框宽度、边框颜色
	图片渲染	• 支持png、jpeg格式图片 • 支持设置至少3种简单滤镜（如灰度、亮度、对比度）
	文本渲染	• 文本元素渲染 • ⭐⭐ 挑战：支持富文本，可设置字体、字号、颜色、背景色

		• ⭐⭐ 挑战：支持加粗、斜体、下划线、删除线
画布交互	无限画布	• 支持画布的自由缩放、拖拽 • ⭐⭐ 挑战：为无限画布实现自定义滚动条 • ⭐⭐⭐ 挑战：实现Minimap（小地图）功能
	选区功能	• 支持单击选中单个元素 • 支持框选同时选中多个元素3. ⭐⭐⭐ 挑战：实现智能对齐的辅助线功能
	快捷键	• 支持选中元素的复制 (Cmd/Ctrl+C) 与粘贴 (Cmd/Ctrl+V)
元素编辑	文本编辑	• 支持双击进入文本编辑模式，进行内容的输入与删除
	基础操作	• 支持对选中元素（单个或多个）进行删除、拖拽、缩放
	旋转	• ⭐ 挑战：支持对选中元素（单个或多个）进行旋转
	组合	• ⭐⭐ 挑战：支持对多个元素进行组合 (Group) 操作，组合可嵌套 • ⭐⭐⭐ 挑战：支持对组合进行打组 (Combine) 与解组 (Break apart)
调参工具栏	浮动工具栏	• 选中不同类型元素（图形、文本、图片）时，出现对应的属性设置面板 • ⭐⭐ 挑战：支持选中部分文本时，工具栏可对局部文字应用样式
性能优化	基础性能	• ⭐⭐ 挑战：画布内承载100个元素，从打开页面到渲染完成，耗时应低于3秒 • ⭐⭐⭐ 挑战：同时操作100个元素（如拖拽、缩放），帧率 (FPS) 保持在50以上

后端部分

作为 MVP 的一部分，你需要实现一套基础的后端服务来支撑画布的核心数据流。

分类	功能点	说明与要求
核心接口	画布 CRUD	• 基于 RESTful 风格设计接口

		• 实现画布文档的创建 (POST canvases)、读取 (GET /canvases/{id}) 和更新 (PUT /canvases/{id})，删除 (DELETE) 可选 • 接口需能接收并返回统一的 Canvas JSON 数据结构
	资源上传	• 实现图片上传接口 (POST /assets/images)，支持 png、jpeg 格式 • 接口应校验 MIME 类型，并建议限制单文件大小（如 5MB） • 成功后返回一个持久化可访问的 URL
	版本与快照	• 实现自动保存（如每次操作后）以生成快照的接口 (POST /canvases/{id}/snapshots) • 提供历史版本列表接口 (GET /canvases/{id}/snapshots) • 提供恢复至指定版本的读取接口 (GET /canvases/{id}/snapshots/{snapshotId})
基础架构	简易鉴权	• MVP 阶段可采用“匿名/本机开发模式”或“单用户固定 Token”方案 • 在说明文档中，需阐述你选择的方案、其安全边界以及未来可如何演进（如引入 OAuth2.0）
	数据持久化	• 可从 MySQL、SQLite、PostgreSQL 或 LowDB（文件型数据库）中选择一种 • 在说明文档中，需阐述你的选择及理由 • 提供可一键启动的脚本或指令（如 npm run start:server），确保服务冷启动后即可运行

P1 级：进阶挑战

完成 P0 后，如果你仍有余力，可以尝试以下挑战，它们将极大提升你作品的竞争力。

分类	功能点	说明与要求
协同与数据	Undo/Redo	• ⭐⭐⭐ 挑战：支持撤销与重做操作 • 建议基于“操作日志” (Command Pattern) 的方式实现，并思考如何处理多用户场景下的冲突最小化
	实时协同	• ⭐⭐⭐⭐⭐ 挑战：支持多人协同编辑 • 需引入 WebSocket，实现当一个用户修改画布时，变更能够实时广播给其他所有打开同一画布的用户
	离线编辑	• ⭐⭐⭐⭐⭐ 挑战：支持断网后的离线编辑

		• 用户在离线状态下仍可操作，网络恢复后，本地的操作队列会自动上报并与服务端数据合并。需简述你实现的冲突处理与合并策略

数据结构与持久化策略建议

画布数据结构

你需要自行设计一套用于描述画布所有元素及其属性的 JSON 结构。这套结构将是前后端通信的“语言”。一个良好的设计应具备可扩展性、可读性，并能清晰地表达元素层级、位置、样式等信息。

持久化策略

- **前端持久化**：对于 MVP，你也可以选择纯前端的持久化方案，如使用 `IndexedDB` 或 `localStorage`。这能让你快速实现“刷新后恢复”的功能。
- **后端持久化**：如 P0 要求所述，通过后端服务进行持久化是更完整的方案。
- **取舍建议**：在说明文档中，请阐述你最终选择的持久化方案。如果同时使用了前后端存储（例如，后端为主，前端为缓存或离线备份），请说明它们之间的数据同步与刷新恢复机制。
- **快照实现**：对于后端版本快照功能，你可以选择“全量快照”（每次保存都存储一份完整的画布 JSON）或“增量操作日志+定期基线”（记录每次操作，定期生成全量快照）的方案。请在文档中说明你的实现选择与考量。

关键指标与实现边界建议

性能指标验证

- **加载时间**：使用浏览器开发者工具的 Performance 面板，记录从页面开始导航到 100 个元素首次绘制完成 (First Contentful Paint, FCP) 的时间。
- **帧率 (FPS)**：同样在 Performance 面板，录制一段你对 100 个元素进行密集操作（如拖拽）的性能剖面，查看其帧率图表是否能稳定在 50 FPS 以上。

交互边界

我们建议你思考并实现以下关键交互的边界情况：

- **选区**：如何处理与部分在视口外的元素的框选？
- **拖拽**：拖拽过程中的性能优化（如使用代理元素）。
- **缩放**：以鼠标指针位置为中心进行缩放。
- **旋转**：旋转操作的控制点 (handle) 设计。
- **组合**：组合后元素的包围盒 (bounding box) 如何计算？其坐标系如何变化？

评审

关注点

将重点从以下几个维度对你的作品进行评估：

- **基础能力的稳定性**：P0 功能是否都已实现，且在常规操作下无明显 Bug。
- **主链路的顺滑度**：核心交互（如拖拽、缩放、编辑）是否流畅，无卡顿。
- **优化策略的可解释性**：你是否在文档中清晰地阐述了你的关键技术决策和优化思路。
- **代码质量与工程规范**：代码结构是否清晰，注释是否得当，Git 提交记录是否规范。

FAQ (常见问题)

挑战项是必须完成的吗？

不是。挑战项属于加分项。我们更看重你对 MVP (P0) 基础能力的扎实实现、稳定的主链路体验，以及在文档中对技术决策的可解释性说明。在保证基础质量的前提下，再去尝试挑战。

持久化必须写后端服务吗？

对于 MVP，你可以选择浏览器本地存储（如 `IndexedDB`）或实现一个简单的后端服务。关键在于刷新页面后，画布内容能够恢复。但在 P0 要求中，我们明确推荐并要求了后端基础功能的实现，以完成更完整的项目闭环。